



GIT

Yohanes Sugiarto

BASIC INTRO TO GIT

Topic:

- The basic Git model
- Pull/clone files from a repository on Github
- Create branch
- Edit files in your own local Git repo
- Push files to a repo on Github

GIT RESOURCES

At the command line: (where verb = config, add, commit, etc.)

```
$ git help <verb>
```

```
$ git <verb> --help
```

```
$ man git-<verb>
```

Free on-line book: <http://git-scm.com/book>

Git tutorial: <http://schacon.github.com/git/gittutorial.html>

Reference page for Git: <http://gitref.org/index.html>

Git website: <http://git-scm.com/>

GIT HISTORY

Came out of Linux development community

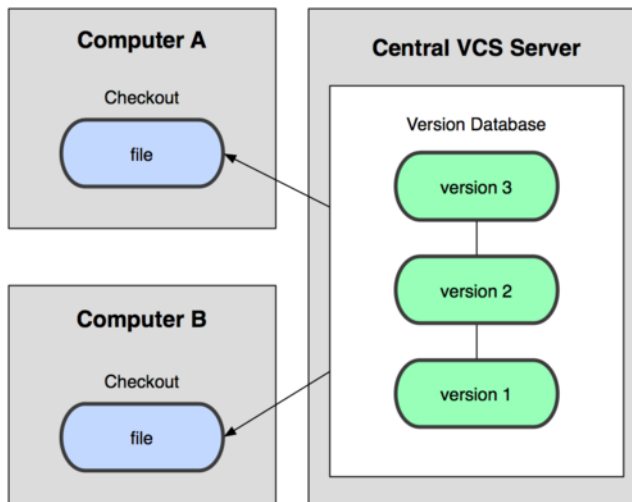
Linus Torvalds, 2005

Initial goals:

- Speed
- Support for non-linear development (thousands of parallel branches)
- Fully distributed
- Able to handle large projects like Linux efficiently

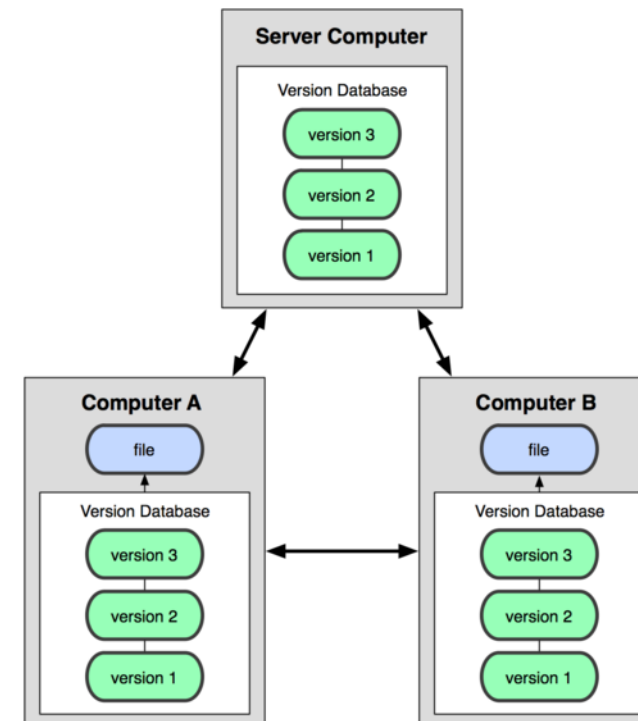
GIT USES A DISTRIBUTED MODEL

Centralized Model



(CVS, Subversion, Perforce)

Distributed Model

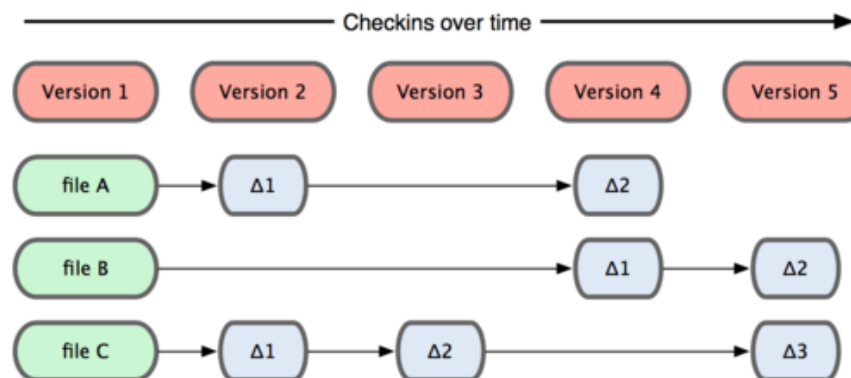


(Git, Mercurial)

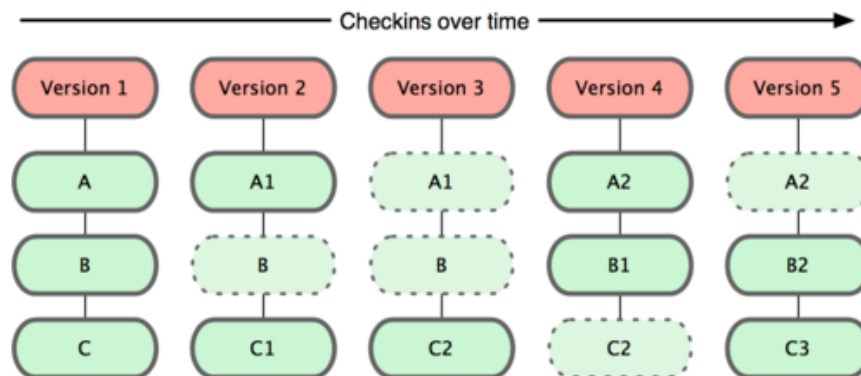
Result: Many operations are local

GIT TAKES SNAPSHOTS

Subversion



Git



GIT USES CHECKSUMS

In Subversion each modification to the central repo incremented the version # of the overall repo.

How will this numbering scheme work **when each user has their own copy of the repo**, and commits changes to their local copy of the repo before pushing to the central server????

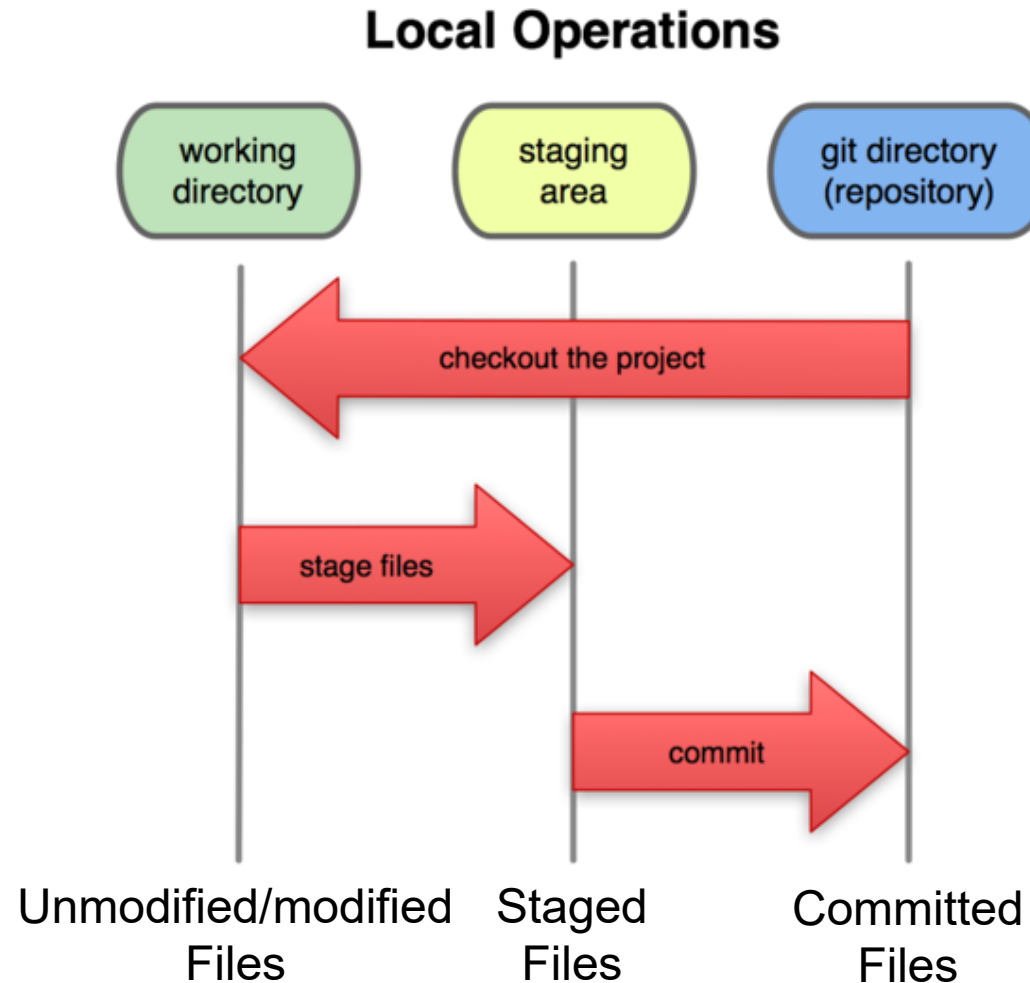
Instead, Git generates a unique SHA-1 hash – 40 character string of hex digits, for every commit. Refer to commits by this ID rather than a version number. Often we only see the first 7 characters:

1677b2d Edited first line of readme

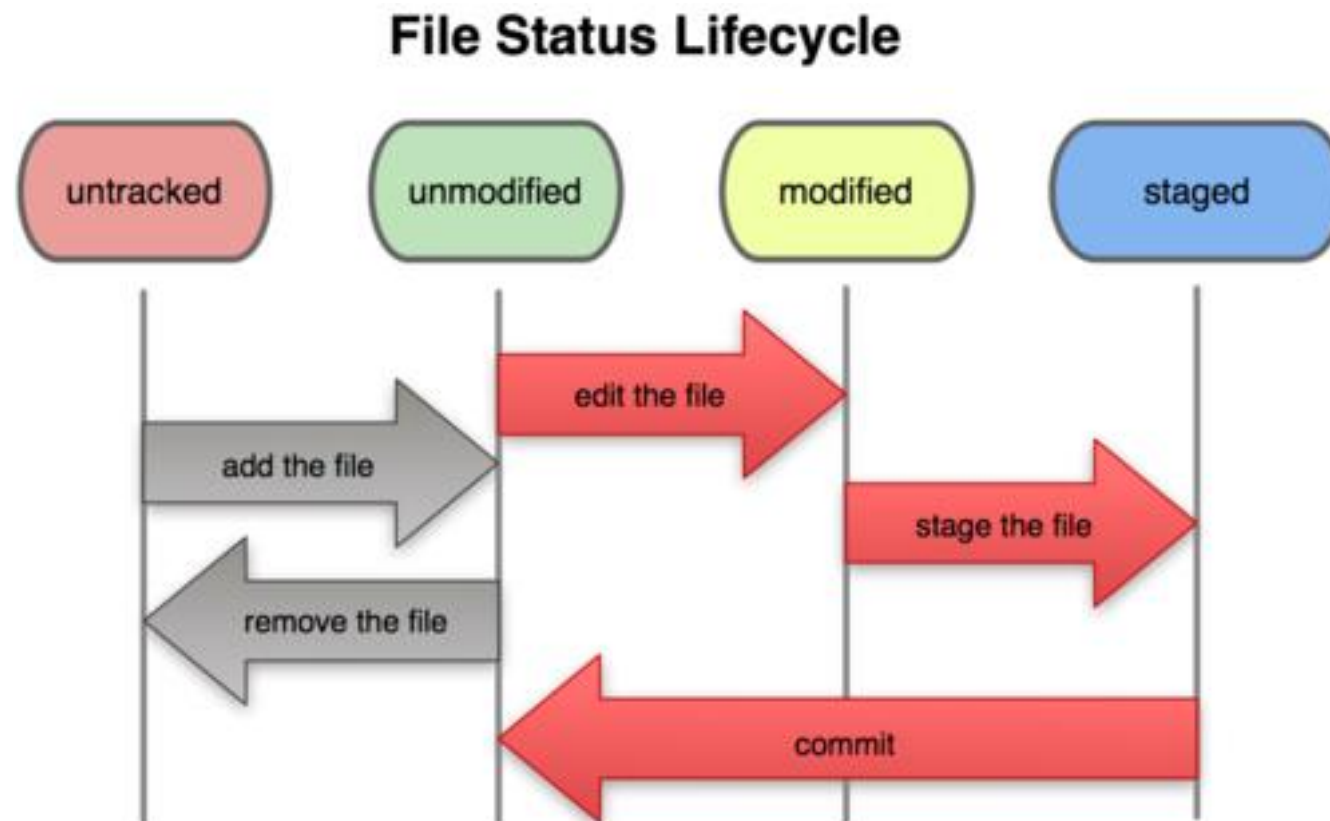
258efa7 Added line to readme

0e52da7 Initial commit

A LOCAL GIT PROJECT HAS THREE AREAS



GIT FILE LIFECYCLE



BASIC WORKFLOW

Basic Git workflow:

1. **Modify** files in your working directory.
2. **Stage** files, adding snapshots of them to your staging area.
3. Do a **commit**, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory.

Notes:

- If a particular version of a file is in the **git directory**, it's considered **committed**.
- If it's modified but has been added to the **staging area**, it is **staged**.
- If it was **changed** since it was checked out but has not been staged, it is **modified**.

GITHUB

[GitHub.com](https://github.com) is a site for online storage of Git repositories.

Many open source projects use it, such as the [Linux kernel](https://www.kernel.org/).

You can get free space for open source projects or you can pay for private projects.

Question: Do I have to use github to use Git?

Answer: No!

- you can use Git completely locally for your own purposes, or
- you or someone else could set up a server to share files, or
- you could share a repo with users on the same file system.

GET READY TO USE GIT!

Set the name and email for Git to use when you commit:

```
$ git config --global user.name "Bugs Bunny"
```

```
$ git config --global user.email bugs@gmail.com
```

- You can call `git config -list` to verify these are set.
- These will be set globally for all Git projects you work with.
- You can also set variables on a project-only basis by not using the `--global` flag.
- You can also set the editor that is used for writing commit messages:

```
$ git config --global core.editor emacs
```

 (it is vim by default)

CREATE A LOCAL COPY OF A REPO

Two common scenarios: (only do one of these)

- a) To clone an already existing repo to your current directory:

```
$ git clone <url> [local dir name]
```

This will create a directory named *local dir name*, containing a working copy of the files from the repo, and a **.git** directory (used to hold the staging area and your actual repo)

- b) To create a Git repo in your current directory:

```
$ git init
```

This will create a **.git** directory in your current directory.

Then you can commit files in that directory into the repo:

```
$ git add file1.java
```

```
$ git commit -m "initial project version"
```

CREATE A LOCAL BRANCH

- Working on your own local branch is highly recommended.
- Command:

```
$ git checkout -b <branch name>
```
- This common practice prevents damaging master branch and keeps the workflow clean.

COMMITTING FILES

The first time we ask a file to be tracked, *and every time before we commit a file* we must add it to the staging area:

```
$ git add README.txt hello.java
```

This takes a snapshot of these files at this point in time and adds it to the staging area.

To move staged changes into the repo we commit:

```
$ git commit -m "Fixing bug #22"
```

Note: To unstage a change on a file before you have committed it:

```
$ git reset HEAD -- filename
```

Note: To unmodify a modified file:

```
$ git checkout -- filename
```

Note: These commands are just acting on your local version of repo.

PULLING FROM & PUSHING FILES TO REPO

Pulling from a repo is a way to get the latest changes:

```
$ git pull origin <branch name>
```

This synchronized your local branch with existing remote/origin branch.

To push your commit on repo:

```
$ git push origin <branch name>
```

GIT COMMANDS

command	description
<code>git clone <i>url</i> [<i>dir</i>]</code>	copy a git repository so you can add to it
<code>git add <i>files</i></code>	adds file contents to the staging area
<code>git commit</code>	records a snapshot of the staging area
<code>git status</code>	view the status of your files in the working directory and staging area
<code>git diff</code>	shows diff of what is staged and what is modified but unstaged
<code>git help [<i>command</i>]</code>	get help info about a particular command
<code>git pull</code>	fetch from a remote repo and try to merge into the current branch
<code>git push</code>	push your new branches and data to a remote repository
others: <code>init</code> , <code>reset</code> , <code>branch</code> , <code>checkout</code> , <code>merge</code> , <code>log</code> , <code>tag</code>	

THANK YOU!